

Non-Slicing Floorplanning-Based Crosstalk Reduction on Gridless Track Assignment for a Gridless Routing System with Fast Pseudo-Tile Extraction

Yu-Ning Chang, Yih-Lang Li, Wei-Tin Lin and Wen-Nai Cheng

Computer Science Department
National Chiao Tung University
Hsin-Chu 300, Taiwan

yuning.cs94g@nctu.edu.tw, {ylli, gis93595}@cs.nctu.edu.tw, winnai.csie89g@nctu.edu.tw

ABSTRACT

Track assignment, which is an intermediate stage between global routing and detailed routing, provides a good platform for promoting performance, and for imposing additional constraints during routing, such as crosstalk. Gridless track assignment (GTA) has not been addressed in public literature. This work develops a gridless routing system integrating a congestion-driven global router, crosstalk-driven GTA and an enhanced implicit connection-graph-based router. Initial assignment is produced rapidly with a left-edge like algorithm. Crosstalk reduction on the assignment is then transformed to a restricted non-slicing floorplanning problem, and a deterministic O-tree based algorithm is employed to re-assign each net segment. Finally, each panel is partitioned into several sub-panels, and the sub-panels are re-ordered using branch and bound algorithm to decrease the crosstalk further. Before detailed routing, routing tree construction is undertaken for placed IRoutes and other pins; many original point-to-point routings are set to connect to IRoutes, and can be accomplished simply with pattern routing. For detailed routing, this work proposes a rapid extraction method for pseudo-maximum stripped tiles to boost path propagation. Experimental results demonstrate that the proposed gridless routing system has over 2.66 times the runtime speedup for fixed- and variable-rule routings of an implicit connection-graph-based router, NEMO. As compared with a commercial routing tool, this work yields an average reduction rate of 15% in coupling capacitance calculated using its built-in coupling capacitance estimator.

Categories and Subject Descriptors: B.7.2 [Hardware]: Integrated Circuits—*Design Aids*: Placement and Routing

General Terms: Algorithms, Design, Performance, Reliability

Keywords: Gridless routing, non-slicing floorplanning, crosstalk reduction, implicit connection graph-based router, full-chip routing, detailed routing

1. INTRODUCTION

Conventional routing flow comprises global and detailed routing [1]. Global routing typically partitions the entire routing region into

sub-regions, i.e., *global cells* (GCells). Every sub-region has a capacity denoting the maximum number of nets that can pass through it. A global router creates a loose route from a subset of global cells connecting the terminals of the net without specifying the actual geometric layout wires. Detailed routing discovers the actual wire segment for every net within the assigned GCells from the global router. In [2], a heuristic algorithm is proposed to optimize the pin alignment across the boundaries of successive GCells. Chang et al. propose a two-stage assignment method (coarse pseudo-pin assignment and detailed pseudo-pin assignment) to control the crosstalk noise with minimizing the cost of wire length and the number of vias [3]. The interconnection complexity finally rises as the feature size continues to scale down, and noise between wires needs to be eliminated. The detailed routing then consumes increasing amounts of time, even with the aid of global routing. Hence, the track assignment (TA), i.e., an intermediate stage between global routing and detailed routing, is incorporated into the two-stage routing flow to lower the routing runtime, and to produce more straight wires than Lee's algorithm [4]. Before the work of [4], the application of TA has been applied in the constrained via minimization to better a completed routing design [5][6][7]. Crosstalk reduction during layer assignment is addressed in [8], where TA is adopted to distribute crosstalk sensitive nets to different layers. In the approach of Batterywala et al. [4], TA only handles nets called *IRoutes* that pass through at least two global cells, and places as many nets as possible on available tracks. *IRoute overlap graph* (OLG) is practically a horizontal constraint graph, and is initially constructed to determine whether two IRoutes can be assigned in a track. The maximum clique in an OLG is identified in order to assign them to different tracks. The assignability of each IRoute to each track is thus denoted by a bipartite graph based on the pre-placed blockages on each track. The TA of the current clique can then be solved using a weighted bipartite matching algorithm. IRoutes are processed clique by clique. TA thus completes the routing of most long nets, and significantly simplifies detailed routing. The research by Ho et al. [9] first involves developing the crosstalk-driven TA in a multilevel routing system. Crosstalk is reduced by identifying a minimum-cost (IRoute overlap length) Hamiltonian path of the currently processed clique obtained from the IRoute OLG. In short, TA provides a good platform for crosstalk reduction in a routing system, and significantly speeds up the routing runtime by 1.3–2.5 times, as revealed in [4].

Conversely, variable width and space interconnection design is a widespread approach for deep submicron (DSM) interconnection optimization in order to reduce the wire resistance and crosstalk effect. Gridless routers construct more complex data structures than grid routers, due to the need to accommodate variable routing rules in the routing graph. Two well-known gridless routers are tile-based and implicit connection-graph-based routers, which possess the

* This work was partially supported by the National Science Council of Taiwan by Grant No. NSC 93-2220-E-009-021, NSC 96-2220-E-009-014, and NSC 96-2220-E-009-011.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ISPD '08, April 13–16, 2008, Portland, Oregon, USA.

Copyright 2008 ACM 978-1-60558-048-7/08/04...\$5.00.

advantages of low path propagation complexity and fast routing graph construction, respectively [10][11][12]. Tile-based routers partition the routing area into a maximum horizontally or vertically stripped tile plane, which is produced by extending horizontal or vertical border lines until it reaches another obstacle or the boundary of the routing region. Tile propagation then proceeds on the tile plane. Implicit connection-graph-based routers extend the border lines of all obstacles to penetrate other obstacles until they reach the boundaries of the routing region. A maze routing algorithm is thus undertaken on the non-uniform grid plane to discover a feasible path. Li et al. [13] present a novel gridless router with the benefits of tile-based and implicit connection-graph-based routers, called NEMO. By considering the implicit connection graph as a tile array instead a grid array, NEMO quickly builds the routing graph with point array, while adjacent and equal-height or -width tiles are grouped as a pseudo-maximum horizontally or vertically stripped tile (PMT) to acquire fast tile propagation. A PMT is dynamically identified when tile propagation explores an unvisited space region, i.e., tile propagation in NEMO includes PMT extraction and propagation. An analysis of routing runtime statistics demonstrates that tile propagation consumes most of the routing runtime, so simplifying PMT extraction is an effective way to accelerate tile propagation in NEMO.

Cong et al. [12] presented a three-stage routing system, comprising performance-driven global routing, congestion-driven wire planning considering variable-rule wires and gridless detailed routing. Wire planning is first applied to all nets one by one before detailed routing to determine the passing regions for each net. Wire planning also helps an incomplete net to refine its new passing regions, by viewing previously completed wires as new obstacles. However, wire planning only determines passing regions of a net instead of its placed track, and processes one net at a time. This work develops an efficient crosstalk-driven gridless routing system, comprising a congestion-driven global router, a crosstalk-driven gridless TA (GTA) and enhanced NEMO with fast PMT extraction. The GTA in this work has two phases, initial GTA and crosstalk reduction. In the first phase, a left-edge like algorithm is invoked to obtain an initial TA result quickly. The problem of re-assigning IRoutes for crosstalk reduction is then transformed into a restricted non-slicing floorplanning problem. A fast deterministic floorplanning algorithm is employed to replace each IRoute in the position with most decreased crosstalk. After refining the position for each IRoute, every panel is treated as a collection of sub-panels, each of which can also be considered as a hybrid IRoute with different states on its top and bottom borders for a horizontal IRoute. Re-ordering the sub-panel can further decrease the total overlapping length. Before detailed routing, routing tree construction is undertaken for placed IRoutes and other pins; many original point-to-point routings are set to connect to IRoutes, and can be completed simply with pattern routing. For detailed routing, this work presents a rapid PMT extraction method to boost path propagation. Experimental results demonstrate that the proposed gridless routing system can perform over 2.66 times faster for fixed- and variable-rule routings than an implicit connection-graph-based router, NEMO. As compared with a commercial routing tool, this work yields an average runtime speedup of 7.92 times and an average 15% reduction rate in coupling capacitance calculated with the built-in coupling capacitance estimator of the commercial routing tool.

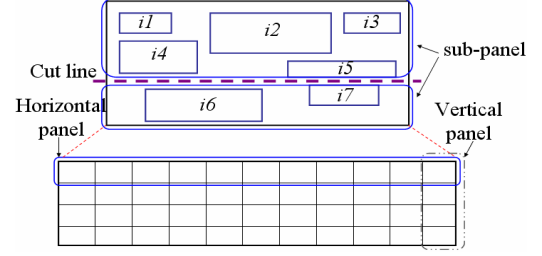


Figure 1. A routing region is partitioned into an 11×4 GCell array; the assignment of the topmost panel with seven IRoutes is zoomed in on the top, and can be divided into two sub-panels with a cut line.

The rest of this paper is organized as follows. Section 2 briefly reviews NEMO, GTA, and crosstalk model. Section 3 presents crosstalk-driven GTA. Section 4 presents the detailed routing. Section 5 discusses the routing results using two rule sets. Finally, Section 6 draws conclusions.

2. PRELIMINARIES

2.1. NEMO Overview

NEMO undertakes routing on an implicit connection plane, which has a much simpler construction process than tile plane construction. During path propagation, NEMO regards implicit connection grid plane as tile plane and groups series of adjacent tiles as PMTs to advance path searching. To validate the legality of every tile in the first time to visit, a slit and interval tree stores all existing blockages, providing an efficient query scheme. If a tile is verified as free, then adjacent free tiles are grouped as a PMT by investigating the slit and interval tree leftwards and rightwards along the leaf nodes and upwards, to search for the blockages stored in unvisited nodes on the upper level. The legality validation and PMT extraction are conducted whenever the router intends to explore a neighboring unvisited region. Since the number of such explorations before reaching the target is very large, the time spent in identifying PMT is a significant issue.

2.2. GTA and Crosstalk Model

Figure 1 depicts a routing example of 11×4 GCells at its bottom of the figure. A panel comprises a series of connected GCells in a row or a column. The topmost horizontal panel contains 7 IRoutes. GTA has no track to be used. One assignment of the topmost horizontal panel is displayed on the top of the figure. The assignment induces one cut line, given by the bold dotted line in Fig. 1, to split the panel into two sub-panels. A cut line is a line not intersecting any IRoute, and can partition IRoutes into two sets; for instance, the cut line in Fig. 1 splits seven IRoutes into two sets, namely five IRoutes for one set and two for the other.

Observations in previous investigations [14][15] reveal that two highly coupling-capacitance related factors are the space and overlapping length between two wires. The impact of wire width has been observed not to be important for variable-width routing [15], indicating that the wire width has only a small influence on coupling effects, with from doubling or tripling the wire width producing a coupling effect variation of about 0.4% to 7%. Consequently, the coupling effect estimation ignores the wire width, and this study adopts the coupling capacitance model used in [16][17][18]. The coupling capacitance between wires i and j is defined as:

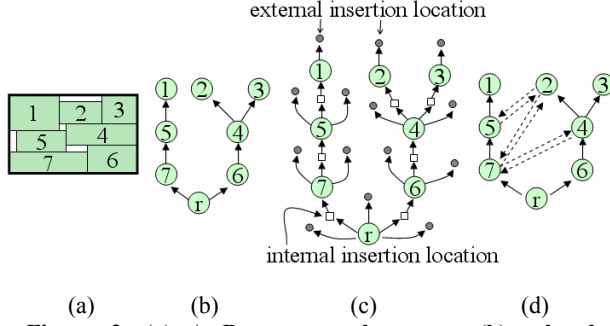


Figure 2. (a) A B-compact placement; (b) related vertical O-tree; (c) external and internal insertion points of the O-tree; (d) extended vertical O-tree integrated with ordered OLG edges.

$$C_c(i, j) = \alpha \cdot f_{i,j} \cdot \frac{l_{i,j}}{d_{i,j}^\beta} \quad (1)$$

where α and β are technology-dependent constants, $l_{i,j}$ is the coupling length, $d_{i,j}$ is the wire spacing between nets i and j and $f_{i,j}$ is the switching factor for nets i and j . $C_c(i, j)$ is observed to drop quickly as two adjacent nets drift apart. The crosstalk model is thus simplified by assuming that two net segments on different layers and in perpendicular directions are not sensitive. Moreover, α and $f_{i,j}$ are ignored and β is assumed to be 2, so the coupling capacitance between two adjacent nets can be estimated simply from $l_{i,j}/d_{i,j}^2$.

The following discussion assumes that the space between any two adjacent IRoutes is fixed.

3. Crosstalk-Driven GTA

The following discussion considers the assignment on a horizontal routing layer. Crosstalk-driven GTA is conducted following global routing, and detailed routing is performed in the last stage to connect all disconnected nets. The entire crosstalk-driven GTA comprises preprocessing, initial assignment and crosstalk reduction. The GTA preprocessor extracts IRoutes for every panel based on the global paths generated by the global router. The IRoute assignment adopts a left-edge like algorithm to yield an initial assignment quickly. The successive floorplanning-based assignment refinement and sub-panel re-ordering diminish the crosstalk impact of the initial assignment. In the following discussion about GTA, the separation rule between any two adjacent IRoutes is defined as sp . All IRoutes are over-sized by $sp/2$ to ensure legal separation between adjacent IRoutes. Therefore, the over-sized IRoute can neighbor on other IRoutes, and two IRoutes whose projection lines can reach each other are considered to induce coupling capacitance. The GTA problem can be considered as a special floorplanning problem once the original IRoutes are over-sized. Every over-sized IRoute is equivalent to a block with a fixed x -coordinate constraint. The objective of this task is to discover a complete floorplanning of the minimum crosstalk within a fixed-height routing region.

3.1. Initial GTA

The initial assignment is intended to place as many IRoutes as possible quickly, while minimizing the length of overlap in the maximum clique. A left-edge algorithm can obtain a utilization-driven initial assignment for grid-based TA. Since the partial assignment of GTA probably has uneven borders, the region cannot easily be regarded as row by row. To ensure compact assignment and crosstalk minimization, the IRoutes of the maximum clique are initially assigned in the order of a minimum-weighted

Algorithm: Extended O-Tree Based Assignment Refinement

Input: Initial track assignment result.

Output: New assignment with reduced crosstalk effect.

begin

1. Construct extended vertical O-tree (EVO_T) for initial assignment;
2. **for** each IRoute Ir {
3. $Min_xtalk = xtalk$ of current assignment;
4. **for** each possible insertion position Ins_pos {
5. $(New_xtalk, new\ assignment) = Tentative_plow(EVO_T, Ir, Ins_pos)$;
6. **if** ($New_xtalk < Min_xtalk$) {
7. $Min_xtalk = New_xtalk$;
8. Set new assignment as the best assignment;
9. }
10. }
11. **if** ($Min_xtalk < xtalk$ of current assignment)
12. replace current assignment with the best assignment;
13. }

end

Figure 3. Algorithm of extended O-tree based

Hamiltonian path to lower the crosstalk effect of the most congested region. A left-edge algorithm is then performed to assign the unfinished IRoutes, starting from the left and right borders of the maximum-clique assignment, and towards the left and right panel boundaries. The unassigned IRoutes on the right side of the maximum-clique assignment are processed in increasing order of their left borders. Every IRoute is assigned to its bottommost available location. Similarly, the unassigned IRoutes on the left side of the maximum-clique assignment are assigned to their bottommost available positions in decreasing order of their right borders.

3.2. Extended O-Tree Based Assignment Refinement (EOBAR)

The crosstalk reduction for a horizontal panel forms a restricted floorplanning problem with a fixed x -coordinate constraint for every IRoute. The floorplanning representation strongly influences the floorplanning performance. O-tree [19] and B*-tree [20] are two well-known methods to represent non-slicing floorplanning. For non-slicing floorplanning, B*-tree is more efficient than O-tree in terms of runtime and area issues. For GTA, the simulated annealing process is not desired, since many panels are involved in decreasing crosstalk, and the total runtime is unacceptable. GTA is not an admissible placement, as revealed by a B-compact placement and its corresponding vertical O-tree in Figs. 2(a) and 2(b). The O-tree of a floorplan can be constructed as follows. Assume that the placement is a B-compact placement, where no block can be shifted downwards from current position with other blocks fixed. A vertical O-tree has a root node on the bottom representing the bottom boundary, and a node representing each block. The root node has an edge directed to the nodes, whose bottom borders are located at the bottom boundary. Any two block nodes, say b_i and b_j , have an edge from b_i to b_j if b_j abuts b_i on its top.

A fast and deterministic method to reduce crosstalk effect is to eliminate each IRoute iteratively, and insert it in another position with crosstalk reduction profit, as applied to non-slicing floorplanning in [19]. Figure 2(c) shows external and internal

insertion positions of the O-tree in Fig. 2(b). All internal insertion locations and the external insertion locations adjacent to the boundary are candidates for altering GTA. For every IRoute, say I_r , all internal and external insertion locations along the O-tree path containing I_r are evaluated to yield the best insertion point to reduce the crosstalk effect.

Algorithm: Tentative_plow

Input: extended O-tree (EOT), inserted IRoute (IRT), insertion location.

Output: the new assignment and its crosstalk effect.

begin

```

1. Identify and store the type-1 SIP in  $SIP\_List$ ;
2. while ( $SIP\_List$  is not empty) {
3.    $SIP = SIP\_List.next()$ ;  $D_{plow} = GetPlowDist(SIP)$ ;
4.   if ( $SIP$  is type 1) {
5.     if ( $IRT$  is above its insertion location)
6.       plow  $SIP$  upwards with a distance of  $D_{plow}$ ;
7.     else { //  $IRT$  is below its insertion location
8.       plow  $SIP$  downwards as far as possible;
9.       if (plowing distance of  $SIP < D_{plow}$ ) {
10.         $SIP = SIP$ 's child in EOT;
11.        plow  $SIP$  upwards with a distance of
12.        ( $D_{plow} - \text{plowing distance of prior } SIP$ );
13.      }
14.    }
15.   Identify and store type-2 SIP in  $SIP\_List$ ;
16. }
17. else //  $SIP$  is type 2
18.   plow  $SIP$  upwards to abut the newly placed IRT
19.   at its top side;
20. } // end of while
21. if (new assignment's height > panel height)
22.   return MAX_INT and no feasible assignment;
23. else {
24.   calculate the crosstalk ( $X_{talk\_new}$ ) of new
25.   assignment;
26.   return  $X_{talk\_new}$  and new assignment;
27. }
28. end

```

Figure 4. Algorithm of IRoute tentative plow.

The EOBAR stage is completed once every IRoute has been evaluated and adjusted to its best location. The EOBAR algorithm is displayed in Fig. 3, where the extended O-tree is introduced later. The procedure *Tentative_plow* in the algorithm realizes the evaluation process of IRoute removal and insertion in the O-tree. The tentative plow removes an IRoute from the O-tree, places it at a specified location, and reports the new crosstalk effect if the new assignment is legal, i.e., if the assignment's height is not greater than the panel's height. To accommodate the inserted IRoute, the tentative plow pushes away the IRoutes that are around the insertion location and overlap the inserted IRoute. The push force may then propagate to the IRoutes on the neighboring O-tree paths until the emptied space can accommodate the inserted IRoute. The notation adopted in the tentative plow is introduced below.

Definition. Interfering IRoute: One IRoute is the interfering IRoute of another IRoute, say I_r , if it has one edge connecting I_r in OLG, and they are not on the same O-tree path. The interfering IRoutes of a plowed IRoute are the potential IRoutes for interfering with the plowing action.

Definition. IRoute under Test (IRT): the IRoute that is removed from the current location and seeks the insertion location with most crosstalk reduction profit.

Definition. IRT path: the O-tree path containing IRT before removing IRT from O-tree.

Definition. Seed IRoute for Plowing (SIP): SIPs are classified into three types. A type 1 SIP is on the IRT path and adjacent to the insertion location. A type 2 SIP is around the insertion location, and overlaps the IRT. A type 3 SIP is the first IRoute imposed by the plowing effect propagating from one O-tree path to another in order to accommodate the inserted IRT. These three types of SIPs and their plow operations are described as follows.

I. Type 1: Two cases are considered in this type. In the first case, the IRT below the insertion location between two IRoutes A and B (where A is on top of B) is removed. In this case, IRoute A is first kept unchanged, and IRoute B is plowed downwards. If IRoute B cannot be shifted down by a distance of the height of IRT, then IRoute A must be also plowed upwards to empty out the space reserved for IRT. The second case eliminates the IRT above its insertion location, which is between two IRoutes A and B (where B is on top of A). In this case, IRoute A remains unchanged, and IRoute B is plowed upwards by a distance of the height of IRT.

II. Type 2: Some IRoutes on the neighboring O-tree paths of IRT path probably overlap IRT after IRT is placed at the insertion location. The IRoutes overlap IRT at the new location must be plowed upwards to abut on the top side of IRT.

III. Type 3: During the upward plow process on the IRT path, an uplifted IRoute B on the IRT path may be hindered by an interfering IRoute C . To accomplish a complete upward plow on the IRT path, IRoute C with the following IRoutes along its O-tree path must also be plowed upwards. IRoute C is called the third-type SIP. Following the plow operation, the O-tree is reconfigured as follows. IRoute B adds a new edge pointing to IRoute C , and IRoute C 's original incident edge is eliminated, i.e., IRoute C finally abuts IRoute B at its top side. Notably, this upward plow effect propagation from one O-tree path to another occurs once an interfering IRoute blocks an uplifted IRoute, since original O-tree is B-compact, and downward plow on a B-compact path is infeasible.

Downward plow is only applied to the type 1 SIP, and the type 2 and 3 SIPs are invoked only by upward plow. Tentative plow first identifies the type 1 SIPs, and then completes associative plow operation. At this time, IRT can be placed at its new location. The type 2 SIP is then identified, and new plow is invoked on the type 2 SIP if IRT overlaps any IRoute on neighboring O-tree path. Figure 4 shows the tentative plow algorithm. For a downward plow, the final location of a falling IRoute is determined by its parent IRoute and the interfering IRoute IIR below and closest to it. If its parent IRoute's location is higher than that of IIR , then the IRoute remains at the top side of its parent IRoute; otherwise, the IRoute moves to the top side of IIR . Since a falling IRoute cannot determine its location until the location of its parent IRoute is fixed, a recursive operation is conducted to finish the downward plow. For an upward plow, the final location of an uplifted IRoute is derived by the desired plowing distance and its parent IRoute's location. Thus a breadth-first search is favorable to upward plow. The lowest interfering IRoute located between original and new locations of an uplifted IRoute initiates a new upward plow on neighboring O-tree path.

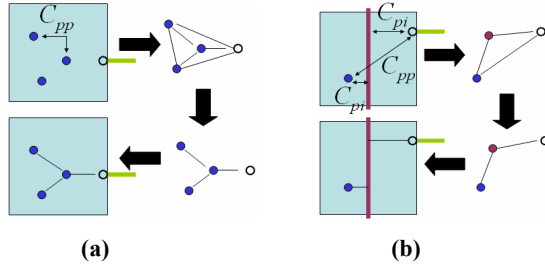


Figure 5. Two edge costs for seeking minimum spanning tree: (a) using the Manhattan distance between any two pins/pseudo pins; (b) using the cost of pin/pseudo pin to IRoute based on their projection distance.

An efficient method for identifying an IRoute's interfering IRoutes is essential to maximizing the performance of a tentative plow. These IRoutes can be identified and access through OLG. However, m insertion points and an OLG of n vertices require accessing $m \times (n-3)$ vertices. To lower the overhead of accessing OLG, the O-tree is integrated with ordered OLG, which is defined as follows. The edges of a vertex are stored in increasing order of IRoute's y -coordinate. The OLG edge, of which one of the two related IRoutes can walk through the directed edges in O-tree to reach the other, is disregarded. The other edges are called *essential*. Figure 2(d) shows the extended O-tree with ordered OLG. The dotted lines denote the essential OLG edges. IRoute 7 has two OLG edges with the order of (7, 4) and (7, 2). Similarly, IRoute 2 has two OLG edges in the order of (2, 7) and (2, 5). An example of removing IRoute 7 and inserting it into the position between IRoutes 1 and 5 is illustrated as follows. In Fig. 2(d), IRoute 7 has two edges in OLG connecting IRoutes 2 and 4, respectively, but no directed path in O-tree connects from IRoute 7 to IRoute 2 or 4, since IRoute 4 is elevated by IRoute 6. The insertion is infeasible if only IRoute 5 is moved down for this insertion, because IRoute 7 overlaps IRoute 4. The correct approach is to move IRoute 5 down along with IRoute 4. IRoute 4 is recognized as a type 2 SIP. Since a type 2 SIP is on a B-compact O-tree path and cannot be plowed downwards, IRoute 4 must be plowed upwards.

3.3. Sub-panel Rearrangement

In the EOBAR stage, crosstalk reduction only considers the effect between individual IRoutes. Further reduction can be attained by considering the effects between sub-panels. In Fig. 1, the assignment is partitioned into two sub-panels with one cut line. Every sub-panel is regarded as a hyper-IRoute (HIR), probably with two different contours. Reordering of these HIRs yields different overlap lengths. The HIR OLG is defined like the IRoute OLG, where every HIR is represented as a node in the graph. There are two directed edges between any two nodes, since HIR is not considered in a mirrored state in this work. A directed edge from a node, says A , to another node, say B , implies that HIR A is placed on the bottom of HIR B . Its edge cost is the overlap length of the assignment sequence $A \rightarrow B$ (where the symbol $\rightarrow$ implies "on bottom of"). The crosstalk minimization problem is then transformed into the Minimum Weighted Hamiltonian Path (MWHP) problem. Finding an MWHP on the clique of IRoute OLG commences with the longest IRoute as the initial node, and selects one adjacent unvisited node with the lowest edge cost out of all the adjacent unvisited nodes. The newly selected node serves as a new initial node, and proceeds similarly until all nodes are visited. This work employs the similar MWHP algorithm, and elevates using the branch and bound algorithm.

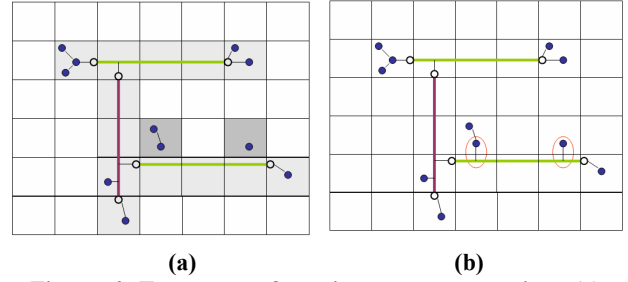


Figure 6. Two steps of routing tree construction: (a) This routing tree is derived after identifying the minimum spanning trees in all GCells with one IRoute passing by; (b) final routing tree is derived after seeking the minimum spanning tree of the routing tree in (a) and all GCells without IRoute passing by.

The branch-and-bound search tree is a multi-ary tree. Each generated node enrolls the total overlap length of the incomplete Hamiltonian path from the root to the present node as the overlap-length lower bound of all paths with the same incomplete partial sequence identified thus far. The sums of all inward edges of all nodes are first computed and sorted in decreasing order to determine the root order of the search trees. The gain of this ordering is avoiding the largest overlap length in the first node probably gets a good assignment, which is a good upper bound for fast dropping worse assignment in successive search. Each node visits its unvisited adjacent node of the least edge cost as the search process proceeds. As the depth-first search continues, the first identified assignment has good quality similar to that in [9], and can behave as a good upper bound for fast convergence. Experimental results indicated that this algorithm can search the optimal solution within 0.001 seconds for most test cases.

4. DETAILED ROUTING

4.1. Routing Tree Construction

Each net routing following GTA leaves several IRoutes and unconnected pins. Planning how to connect pins and IRoutes together is stipulated to achieve good routing quality. Generally, several IRoutes are derived based on the global path. Two pseudo pins are defined for each IRoute in the two ending GCells of a straight global path. The remaining work for detailed net routing is to connect all pins, pseudo pins and IRoutes together. Figure 5 shows two cases of routing tree construction. In Fig. 5(a), all pins and pseudo pins are regarded as nodes, and any two nodes have an edge with a cost defined by their Manhattan distance. This cost, called the Manhattan distance cost, of any two pins or pseudo pins, is denoted as C_{pp} . The other edge cost defines the edge cost of pin or pseudo pin to IRoute. Figure 5(b) displays a case of a GCell containing a pin, a pseudo pin and an IRoute. An IRoute is also denoted as a node in the complete graph, but its edge cost, represented as C_{pi} , between itself and any pin or pseudo pin is shortest distance between them. For instance, the edge cost between the IRoute and the pin or pseudo pin in Fig. 5(b) is given by the horizontal distance from the pin or pseudo-pin to the projection point on the IRoute. Prim's algorithm is adopted to seek the minimum spanning tree. This routing tree determines the point-to-point routings invoked by detailed routing.

The routing tree found at this point probably does not include all pins of a net, since some segments are too short to be processed by GTA. For instance, there are two short segments not processed by GTA, as shown with two heavy shadow GCells in Fig. 6(a), and its routing tree is not complete.

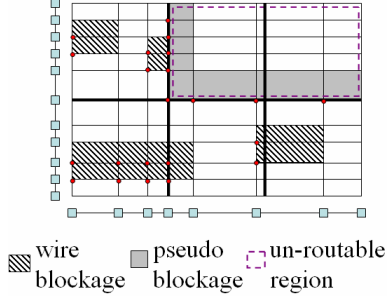


Figure 7. This routing region contains 3×2 GCells (bins). The global path is the L-shape, and the right two GCells on top row are un-routable. All tiles within obstacles are marked as blocked by placing a circle on the left bottom corner in the figure. PMT extraction only requires checking the states of the queried tile and its left/bottom and right/top successive tiles for a horizontal/vertical layer graph.

To complete the entire routing tree construction, the routing tree identified so far, and all pins in a short-segment GCell, are tree identified so far, and all pins in a short-segment GCell, are treated as nodes, and the edge cost of a pin to the existing routing tree is the shortest distance between them. For each short-segment GCell, a complete graph is constructed and minimum spanning tree is discovered. The final routing tree is then completed, as shown in Fig. 6(b).

4.2. Pattern Routing

Pattern routing has already been adopted in global routing to increase the routing speed [21][22]. This work conducts pattern routing before detailed routing to simplify some easy routings. Detailed routing can solve all point-to-point routings after building the routing tree. However, the point-to-point routings are totally different from those in a two stage routing flow, i.e., global routing plus detailed routing. Long-distance routing has been completed by GTA, and the detailed router is responsible for short-distance routing and incomplete IRoutes. Observations on some real circuits, many short-distance routings are based on pin-to-IRoute routing. Most such routings can be completed using a direct or L shape connection. Therefore, simple direct connection and L-shape routing is undertaken at this stage.

4.3. Bin-Based Data Structure and Fast PMT Extraction

The detailed routing is dealt with by NEMO in this work. NEMO performs gridline reduction to construct a simplified routing graph. Gridline reduction tends to impose a searching penalty as the number of obstacles inducing a gridline rises. The bin-based data structure has been widely adopted in physical design. If the bin size is set equal to the GCell size, then the bin-based storing scheme simplifies gridline generation, since gridlines are naturally reduced by simply extending the borders of obstacles in the GCells (bins) of the global path. Figure 7 shows a routing region comprising 3×2 GCells (bins). The bold lines define GCells and bins. The global path is an L-shape, and the rightmost two GCells on the top row are not allowed for this routing. To speed up PMT extraction, pseudo-blockages are inserted around the global path to restrict identified PMTs within the global path. Moreover, NEMO dynamically identifies PMTs during path propagation. Although the slit and interval tree is an efficient data structure, it inevitably requires numerous queries before reaching the target. Hence, if PMTs can be identified without visiting the slit and interval tree, then path

Table 1. Comparison of routing performance between NEMO and this work.

	NEMO		This work	
	Time (Tn:sec)	Mem (MB)	Time (Tt:sec)	Mem (MB)
s5378	2.4	10	2.10	22
s9234	1.7	9	1.45	21
s13207	6.6	15	4.43	25
s15850	8.8	18	6.51	27
s38417	37.2	48	13.46	37
s38584	73.7	66	30.52	45
Tn/Tt	1.72		1	

Table 2. Statistics of crosstalk reduction for fixed- and variable-rule routings.

Circuit	Initial assignment		O-tree based refinement + HIR re-arrangement			
	Coupling cap. $\times 10^3$ (C1)		Coupling cap. $\times 10^3$ (C2)		RR (%)	
	FR	VR	FR	VR	FR	VR
S5378	.168	.123	.075	.091	55	26
S9234	.107	.086	.040	.047	63	46
S13207	.379	.294	.160	.205	58	30
S15850	.493	.363	.205	.275	59	24
S38417	1.013	.794	.385	.559	62	30
S38584	1.402	1.026	.578	.863	59	16
Ave.					59	29

· Coupling cap.: $l_{i,j}/d_{i,j}^2$; FR: fixed rule; VR: variable rule;

· RR: reduction rate = $(C1-C2)/C1$;

propagation can be performed more efficiently than previously. Since the point array has been established, as indicated by the series of rectangles on the bottom and left side of the routing region, all bins are scanned sequentially to set as *blocked* the tiles covered by obstacles within each bin. All tiles outside and neighboring the global path are also set as blocked. In Fig. 7, the tiles whose left bottom corners are marked with a circle are set blocked. Identifying PMTs becomes simple and straightforward once all tiles within obstacles are set blocked and the others are set *unvisited*. For a horizontal layer graph, if a queried tile is in the unvisited state, then PMT extraction groups its left and right successive unvisited tiles until a routing boundary or blocked tile are reached. This operation is fairly simple.

5. Experimental Results

All routing tests were conducted on a 1.2GHz Sun Blade-2000 workstation with 2GB memory with six MCNC benchmark circuits. All cases were routed using three routing layers.

5.1. Enhanced Implicit Connection-Graph-Based Routing

To determine the efficiency of the proposed method for improving NEMO, all cases were routed by global and detailed routers. Detailed routing was completed with NEMO and the proposed enhanced NEMO. Table 1 displays the routing performance comparison between NEMO and this work. The memory required by bin-based data structure increased slowly to gain the superiority over slit and interval tree for the last two circuits. Meanwhile, the enhanced router had 1.72 times the runtime speedup of NEMO on average. Since the proposed method only adjusts the

methods of gridline extraction and PMT extraction, all routing cases are finished with the same routing quality as NEMO.

5.2. Gridless Routing System

The entire routing system comprises a global router, crosstalk-driven GTA, and enhanced NEMO. Table 2 shows the crosstalk reduction, for fixed- and variable-rule routings in the GTA stage. Variable-rule routing was performed using the same circuits with modified rule set as follows. The width of the longest 10% of nets was doubled; that of the next 10% of nets was multiplied by 1.5, while the others remain unaltered. Since many pins on the first metal layer are aligned and set apart in minimum-rule space, and design rule violation occurs if the rule of the first metal layer and its pins are widened, the rules of the first metal layer remained unchanged. GTA comprises three stages, initial assignment, O-tree based iterative re-assignment and HIR reordering. In Table 2, columns 2 and 4 compare the coupling capacitance of initial assignment and GTA's result for fixed-rule routing, and columns 3 and 5 list the coupling capacitance of initial assignment and GTA's result for variable-rule routing. For fixed-rule routing, GTA contributes from 55–63% coupling capacitance reduction (59% on average), while a coupling capacitance reduction of 16–46% is achieved by GTA for variable-rule routing (29% on average). Although initial assignment does not totally focus on crosstalk reduction, and may cause bad-quality assignment in terms of coupling capacitance. Three-phase algorithm still produces a fast and efficient crosstalk-driven GTA.

Table 3 compares the routing statistics of this work and that of Li et al. [13] for fixed- and variable-rule routings. The runtime for this work is split into five stages, namely initialization for GTA, GTA, preprocessing before detailed routing, pattern routing and detailed routing. The runtime speedup for six test cases is in the range 1.4–3.3 times and 1.5–5.2 times for fixed- and variable-rule routings, respectively. This work gains a runtime speedup of 2.66 and 4.17 times on average for fixed- and variable-rule routings. However, the wire length is getting longer by approximately 6% for crosstalk minimization for both rule sets. All cases were 100% completed under two rule sets.

Six test cases are also routed with a commercial routing tool enabling signal integrity (SI) capability. Table 4 compares the routing results and coupling capacitance of this work and that of the commercial routing tool for fixed-rule routing. The runtime speedup for six test cases is in the range 6.93–11.2 times (7.92 times speedup on average), while the total wire length has about 4% increasing rate. As regarding the comparison of coupling capacitance, the routing results of this work are translated to DEF files and then fed into the commercial routing tool. The built-in coupling capacitance estimator of the commercial routing tool is employed to calculate the coupling capacitance of this work and that of the commercial routing tool. This work yields less coupling capacitance than the routing tool in all cases with the reduction rate in the range 8–18% (15% on average). The commercial routing tool split a long net into several wire segments routed in different tracks, however the proposed track assignment algorithm does not consider wire breaking currently. Wire breaking probably provides better connections of pins to IRoutes, which results in less wire length. Besides, wire breaking can contribute to the further reduction of coupling capacitance.

6. Conclusions

This work presents a three-stage gridless routing system, comprising a congestion-driven global router, a crosstalk-driven GTA and an enhanced implicit connection-graph-based detailed router. Crosstalk reduction in GTA is transformed to a non-slicing floorplanning problem, and an O-tree-based deterministic floorplanning algorithm is employed. Further reduction is obtained

by splitting the routing region into HIRs, then re-ordering the HIRs with a branch and bound algorithm. After GTA and routing tree construction, many original point-to-point routings are set to connect to IRoutes, and can be simply resolved using pattern routing. Finally, the detailed router simplifies its gridline extraction and PMT extraction by adopting a bin-based data structure and tagging blocked tiles. Experimental results reveal that the proposed gridless routing system can perform over 2.66 times faster for fixed- and variable-rule routings than an implicit connection-graph-based router, NEMO. As compared with a commercial routing tool enabling signal integrity optimization, this work yields an average runtime speedup of 7.92 times and an average 15% reduction rate in coupling capacitance calculated with the built-in coupling capacitance estimator of the commercial routing tool, with 4% wire-length increasing rate. For further crosstalk and wire-length reduction, wire breaking has to be considered in track assignment to develop the flexibilities of lowering overlap length of adjacent wires and of connecting pins to IRoutes.

7. References

- [1] S. M. Sait and H. Youssef, "VLSI physical design automation," World Scientific Publishing, 1999.
- [2] W.-C. Kao and T.-M. Parn, "Cross point assignment with global rerouting for general-architecture designs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 14, no. 3, pp. 337–348, Mar. 1995.
- [3] C.-C. Chang and J. Cong, "Pseudopin assignment with crosstalk noise control," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 5, pp. 598–661, May 2001.
- [4] S. Batterywala, N. Shenoy, W. Nicholls, and H. Zhou, "Track assignment : a desirable intermediate step between global routing and detailed routing," *IEEE International Conference on Computer Aided Design*, pp. 59–66, Nov. 2002.
- [5] Y. S. Kuo, T. C. Chern, W.-K. Shih, "Fast algorithm for optimal layer assignment," *Proceedings of the 25th ACM/IEEE conference on Design automation*, pp. 554–559, 1988.
- [6] C.-J. R. Shi, "Solving constrained via minimization by compact linear programming," *Proceedings of the Asia and South Pacific Design Automation Conference*, pp. 635–640, 1997.
- [7] C.-C. Chang and J. Cong, "An efficient approach to multilayer layer assignment with an approach to via minimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 18, no. 5, pp. 608–620, May 1999.
- [8] J. D. Cho, S. Raje, M. Sarrafzadeh, M. Sriram, and S. M. Kang, "Crosstalk-minimum layer assignment," *Custom Integrated Circuits Conference*, pp. 29.7.1–29.7.4, 1993.
- [9] T.-Y. Ho, Y.-W. Chang, S.-J. Chen, and D. T. Lee, "Crosstalk- and performance-driven multilevel full-chip routing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 24, No. 6, pp. 869–878, June 2005.
- [10] A. Margarino, A. Romano, A. De Gloria, F. Curatelli, and P. Antognetti, "A tile-expansion router," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. CAD-6, pp. 507–517, July 1987.
- [11] Jeremy Dion and Louis M. Monier, "Contour: A Tile-based Gridless Router," Western Research Laboratory Research Report 95/3, Palo Alto, California.
- [12] J. Cong, J. Fang, and K. Khoo, "DUNE: A multilayer gridless routing system," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 20, No. 5, pp. 633–646, May, 2001.
- [13] Yih-Lang Li, Xin-Yu Chen, and Zhi-Da Lin, "NEMO: A New Implicit Connection Graph-based Gridless Router with Multi-layer Planes and Pseudo-tile Propagation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 26, No. 4, pp. 705–718, Apr. 2007.

- [14] S.-W. Tu, W.-Z. Shen, Y.-W. Chang, T.-C. Chen, and J.-Y. Jou, "Inductance modeling for on-chip interconnects," *International Journal of Analog Integrated Circuits and Signal Processing*, Vol. 35, No. 1, pp. 65–78, April 2003.
- [15] L. He and M. Xu, "Modeling and Layout Optimization for On-Chip Inductive Coupling," *U. of Wisconsin at Madison, Technical Report ECE-00-1*, Dec 1999.
- [16] H. Zhou and D. F. Wong, "Global routing with crosstalk constraint," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 18, No. 11, pp. 1683–1688, 1999.
- [17] H.-P. Tseng, L. Sheffer and C. Sechen, "Timing- and crosstalk-driven area routing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 20, No. 4, pp. 528–544, 2001.
- [18] Di Wu, J. Hu, M. Zhao and R. Mahapatra, "Timing driven track routing considering coupling capacitance," *Proceedings of the Asia and South Pacific Design Automation Conference*, pp. 1156–1159, 2005.
- [19] Pei-Ning Guo, Chung-Kuan Cheng, and Takeshi Yoshimura, "Floorplanning Using a Tree Representation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 20, No. 2, pp. 281–289, 2001.
- [20] Y.C. Chang, Y. W. Chang, G. M. Wu, and S. W. Wu, "B*-trees : A New Representation for Non-slicing Floorplans," *Proceeding of ACM/IEEE Design Automation Conference*, pp. 458–463, June 2000.
- [21] R. Kastner, E. Bozozzadeh, and M. Sarrafzadeh, "Predictable routing," *Proceeding of IEEE/ACM Intl. Conf. Computer-Aided Design*, pp. 110–113, 2000.
- [22] Min Pan and Chris Chu, FastRoute, "A Step to Integrate Global Routing into Placement," *Proceeding of IEEE/ACM International Conference on Computer-Aided Design*, pages 464–471, 2006.

Table 3. Comparison of routing results between this work and [13].

	Three-stage gridless routing system															
	Run time (sec)														W.L. ($\times 10^4 \mu\text{m}$)	
	Ini. (1)		GTA C.R. (2)		Preprocess (3)		Pattern routing (4)		(1)+(2)+(3) +(4)		Enhanced NEMO		total			
	FR	VR	FR	VR	FR	VR	FR	VR	FR	VR	FR	VR	FR	VR	FR	VR
s5378	0.05	0.04	0.63	0.85	0.11	0.07	0.07	0.06	0.86	1.02	0.84	1.29	1.70	2.31	8.0	8.2
s9234	0.04	0.02	0.38	0.38	0.09	0.07	0.06	0.05	0.57	0.52	0.68	1.2	1.25	1.72	6.0	6.1
s13207	0.13	0.09	1.7	1.72	0.25	0.17	0.24	0.19	2.32	2.17	1.81	5.45	4.13	7.62	19	19
s15850	0.16	0.09	2.26	2.97	0.29	0.21	0.30	0.23	3.01	3.5	2.33	4.83	5.34	8.33	23	24
s38417	0.43	0.29	6.42	5.74	0.81	0.71	0.91	0.65	8.57	7.39	5.47	8.36	14.04	15.75	52	52
s38584	0.62	0.40	10.52	11.24	2.04	1.53	1.60	1.12	14.78	14.29	7.87	13.35	22.65	27.64	72	72
Comp.													1	1	1	1

	Global Routing + NEMO			
	Run time (sec)		W.L. ($\times 10^4 \mu\text{m}$)	
	FR	VR	FR	VR
s5378	2.4	3.74	7.4	7.6
s9234	1.7	2.69	5.5	5.6
s13207	6.6	11.28	17	18
s15850	8.8	16.33	22	22
s38417	37.2	56.36	48	49
s38584	73.7	143.39	67	68
Comp.	2.66	4.17	0.93	0.94

· **Ini.**: initialization for GTA; **GTA C.R.**: three-phase GTA crosstalk reduction; **Preprocess**: preprocessing for detailed routing.

Table 4. Comparison of fixed-rule routing results of a commercial routing tool and this work.

circuit	Run time (sec)		Wire length ($\times 10^4 \mu\text{m}$)		Coupling capacitance (pf)	
	This work	CR wt SI	This work	CR wt SI	This work	CR wt SI
s5378	1.70	16	8.0	7.7	3.88	4.67
s9234	1.25	14	6.0	5.7	2.28	2.72
s13207	4.13	43	19	18	8.38	9.74
s15850	5.34	49	23	23	11.29	13.77
s38417	14.04	110	52	50	21.78	23.68
s38584	22.65	157	72	69	32.69	37.06
Comp.	1	7.92	1	0.96	1	1.17

· CR wt SI: commercial routing tool with signal integrity capability.